

Clasificador de Géneros Musicales

con Inteligencia Artificial

Guía de estudio para la defensa final

Análisis de audio con librosa, SVM y Redes Neuronales

Este documento explica **en profundidad** cómo funciona cada etapa del proyecto: desde la física del sonido y cómo librosa extrae las características, hasta por qué los modelos asocian el folklore con *country* o *música clásica*.

Índice

1. Introducción y objetivos	2
1.1. Los dos experimentos	2
1.2. Pipeline general	2
2. Estado del proyecto: modelos, datos y archivos	3
2.1. El flujo de datos	3
2.2. Los modelos guardados (y por qué son 3 archivos cada uno)	3
2.3. ¿Hace falta el audio una vez que tengo el CSV?	4
2.4. Resumen del estado actual	4
3. Cómo funciona librosa en profundidad	4
3.1. Qué es una señal de audio digital	4
3.2. El paso clave: de la onda al espectro (STFT)	5
3.3. El truco mean/std: de una matriz a un número	5
4. Las 45 features, una por una	6
4.1. MFCC — la feature estrella (análisis en profundidad)	6
4.1.1. ¿Qué problema resuelven?	6
4.1.2. Cómo se calculan (paso a paso)	6
4.2. Chroma — la armonía y las notas	7
4.3. Features espectrales — el “color” del sonido	7
4.4. Features temporales — ritmo y energía	7
4.5. Por qué se normaliza (StandardScaler)	7
5. Modelo 1: SVM (Support Vector Machine)	8
5.1. Idea intuitiva	8
5.2. El kernel RBF: por qué es no lineal	8
5.3. Multiclase: 10 géneros con un clasificador binario	8
5.4. Resultados del SVM (GTZAN)	8
6. El corazón del proyecto: por qué el modelo confunde el folklore	9
6.1. Experimento A: folklore sin entrenar	9
6.2. Experimento B: agregar folklore como género nuevo	9
6.2.1. La primera versión (ingenua) y su problema	9
6.2.2. La versión correcta: cross-validation 5-fold	10
6.2.3. Resultados reales	10
7. Modelo 2: Red Neuronal (MLP)	11
7.1. Anatomía de la red	11
7.2. Cómo aprende	11
7.3. Comparación cualitativa	12
7.4. Comparación justa (mismo dataset y mismo split)	12
8. Modelo 3 (descartado): CNN sobre espectrogramas	13
9. Preguntas probables en la defensa (y cómo responderlas)	13

1. Introducción y objetivos

El proyecto es un **clasificador automático de géneros musicales**. La idea central es: tomar un archivo de audio, convertirlo en una lista de números que describen sus propiedades acústicas (las *features*), y entrenar un modelo de Machine Learning que aprenda a asociar esos números con un género.

La pregunta de investigación

No buscamos solo “acertar el género”. Lo interesante es **entender por qué** el modelo decide lo que decide. En particular: ¿qué pasa cuando le mostramos folklore argentino, un género que el modelo nunca vio? ¿Por qué lo confunde con *country* o *clásica*?

1.1 Los dos experimentos

Experimento A — Folklore *sin* entrenar

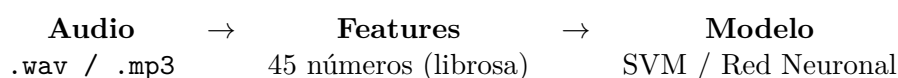
Entrenamos un SVM solo con el dataset GTZAN (10 géneros internacionales) y luego le pedimos que clasifique canciones de folklore argentino. Como el modelo no conoce “folklore”, se ve *obligado* a encajarlo en alguno de los 10 géneros que sí conoce. El resultado revela qué géneros “suenan parecido” al folklore según las features.

Experimento B — Folklore *con* entrenamiento

Agregamos 70 canciones de folklore como un género nuevo (#11), reentrenamos y evaluamos con *cross-validation* sobre canciones que quedaron fuera del entrenamiento. El modelo reconoce el folklore ~66 % de las veces: demuestra que las features **sí** capturan la identidad del género, solo faltaban ejemplos.

Adicionalmente se entrena una **Red Neuronal** (perceptrón multicapa) sobre las mismas features para comparar con el SVM, y existe un experimento con **CNN** sobre espectrogramas (descartado en principio, pero documentado al final).

1.2 Pipeline general

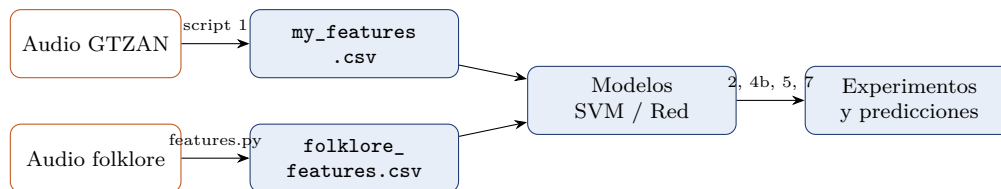


Script	Etapas	Qué hace
features.py	Módulo común	Funciones compartidas: extract_features y la carga del folklore con cache
1_data_extraction.py	Extracción	Recorre GTZAN, extrae 45 features por canción → my_features.csv
2_model_training.py	Entrenamiento	Entrena SVM (RBF), guarda modelo + scaler + label encoder
3_folklore_analysis.py	Experimento A	Predice folklore con el SVM sin reentrenar
4_folklore_training_and_testing.py	Experimento B	Reentrena con folklore incluido y prueba con 1 canción no vista
4b_folklore_crossvalidation.py	Experimento B (v2)	Evalúa el folklore con <i>cross-validation</i> 5-fold (versión correcta)
5_neuronal_network.py	Comparación	Red neuronal sobre GTZAN (10 géneros)
6_cnn_training.py	Opcional	CNN sobre mel-espectrogramas (descartado)
7_model_comparison.py	Comparación justa	SVM vs Red Neuronal sobre el mismo split (11 clases)

2. Estado del proyecto: modelos, datos y archivos

Antes de entrar en los detalles, este es el “mapa” del proyecto: qué se genera, qué modelos quedan guardados y qué archivos hacen falta para cada cosa.

2.1 El flujo de datos



Hay **dos** fuentes de audio que se extraen **por separado**, y este es un punto que suele confundir:

- **GTZAN**: lo extrae `1_data_extraction.py` una sola vez → `my_features.csv` (999 canciones, 45 features c/u).
- **Folklore**: el script 1 **no** lo procesa. Lo cargan los scripts de folklore (3, 4, 4b y 7) a través de `features.py`, que extrae desde la carpeta de folklore **una sola vez** y guarda `folklore_features.csv` (70 canciones). En las corridas siguientes se lee del cache.

Una vez que las features están en los CSVs, el entrenamiento y los experimentos (scripts 2, 5, 7 y la parte de modelado de 4b) ya **no** tocan el audio.

Extracción centralizada (refactor)

La función `extract_features` estaba **copiada** en varios scripts, y los scripts 3 y 4 re-extraían el folklore en cada corrida. Se centralizó todo en el módulo `features.py`: una única `extract_features` y una función `load_folklore_features` con cache. Ahora todos los scripts importan de ahí, no hay código duplicado y el audio del folklore se procesa una sola vez.

2.2 Los modelos guardados (y por qué son 3 archivos cada uno)

Algo que confunde al principio: **cada “modelo” son en realidad 3 archivos** que viajan juntos. No alcanza con el clasificador solo; para predecir el género de una canción se necesitan los tres:

Archivo	Qué es	Para qué sirve
<code>svm_*.pkl</code>	Clasificador SVM	Hace la predicción del género
<code>scaler_*.pkl</code>	StandardScaler	Normaliza las features de una canción nueva <i>igual</i> que en el entrenamiento. Sin esto, los números entran en otra escala y el modelo falla
<code>label_encoder_*.pkl</code>	LabelEncoder	Traduce entre el número interno del modelo y el nombre del género (0 ↔ blues, 4 ↔ folklore, ...)

En la carpeta `models/` hay **dos modelos**, ambos SVM:

Modelo (trío)	Clases	Qué hace
<code>svm_model</code> (+ scaler, encoder)	10 (GTZAN)	Clasificador base. Es el del Experimento A : predice folklore con géneros que ya conoce (→ <i>country</i>)
<code>svm_with_folklore</code> (+ ...)	11 (GTZAN + folk)	El que sí conoce “folklore” como clase. Modelo final del Experimento B

La red neuronal y la CNN no se guardan en disco

Los únicos modelos persistidos son los **dos SVM**. La **red neuronal** se entrena en memoria dentro de los scripts 5 y 7 (se usa y se descarta en cada corrida), y la **CNN** fue descartada y removida del repositorio (queda solo `6_cnn_training.py` como referencia). Esto es a propósito: el foco del proyecto es el *análisis*, no servir predicciones en producción.

2.3 ¿Hace falta el audio una vez que tengo el CSV?

Para reproducir los experimentos y entrenar: no. Los scripts 2, 4, 4b, 5 y 7 solo leen los CSVs, nunca abren un `.wav`. Con los dos CSVs alcanza para reentrenar todo y reproducir cada resultado.

El audio sí se necesita en dos casos:

1. **Re-extraer features:** si cambiás qué features usás, o agregás/quitás canciones, hay que volver a correr la extracción sobre los `.wav`.
2. **Clasificar una canción nueva:** el modelo no “come” audio, come el vector de 45 números. Para una canción que no está en el CSV, primero hay que extraerle las features desde su `.wav`.

En resumen, el audio pasa a ser **materia prima**: para lo ya hecho no se necesita; para extender el dataset o probar canciones nuevas, sí.

2.4 Resumen del estado actual

Componente	Estado	Detalle
Extracción de features (script 1)	✓	45 features con librosa
SVM base (10 géneros)	✓	~68 % accuracy
SVM con folklore (11 clases)	✓	guardado en <code>models/</code>
Experimento A (folklore sin entrenar)	✓	cae en <i>country</i>
Experimento B + cross-validation	✓	recall 65.7 % $\pm$ 9.5 %
Red neuronal + comparación justa	✓	empata con SVM (~66–68 %)
Documentación (este PDF)	✓	guía de estudio completa
CNN	descartada	solo queda el script de referencia

3. Cómo funciona librosa en profundidad

Este es uno de los puntos que el profesor remarcó: entender de verdad qué hace la librería, no usarla como una caja negra.

3.1 Qué es una señal de audio digital

El sonido es una onda de presión que varía en el tiempo. Para procesarla en una computadora se **muestrea**: se mide la amplitud de la onda muchas veces por segundo. Ese ritmo de muestreo es el **sample rate** (*sr*).

```
1 y, sr = librosa.load(file_path, duration=30, sr=22050)
```

- `y` es un arreglo de NumPy con las amplitudes de la onda. Si cargamos 30 segundos a 22050 Hz, y tiene $30 \times 22050 = 661,500$ números.

- `sr = 22050` significa **22050 muestras por segundo**. `librosa` usa este valor por defecto (la mitad del estándar de CD, 44100 Hz). Es suficiente porque por el *teorema de Nyquist* permite representar frecuencias hasta $sr/2 = 11025$ Hz, que cubre casi toda la energía musical relevante.
- `duration=30` recorta a los primeros 30 segundos (igual que GTZAN, donde cada clip dura 30 s).

¿Por qué fijar el sample rate?

Si distintas canciones se cargaran a distintos *sample rates*, las features no serían comparables. Fijar `sr=22050` garantiza que todas las canciones se analizan en la misma “resolución temporal”.

3.2 El paso clave: de la onda al espectro (STFT)

La onda cruda (dominio del tiempo) dice *cuándo* suena fuerte, pero no *qué frecuencias* contiene. Para eso se aplica la **Transformada de Fourier de Tiempo Corto (STFT)**:

1. Se parte la señal en **ventanas** cortas que se solapan (típicamente de 2048 muestras, avanzando de a 512 — el `hop_length`).
2. A cada ventana se le aplica la **Transformada de Fourier**, que la descompone en sus frecuencias componentes (¿cuánta energía hay en los graves, en los medios, en los agudos?).
3. El resultado es un **espectrograma**: una matriz *frecuencia* $\times$ *tiempo* donde cada celda dice cuánta energía hay en esa frecuencia en ese instante.

Idea central

Casi todas las features de `librosa` (MFCC, chroma, centroide, rolloff, bandwidth) se calculan **a partir del espectrograma**. Entender el espectrograma es entender el 80% de la librería.

3.3 El truco mean/std: de una matriz a un número

Un espectrograma de 30 segundos tiene cientos de columnas (una por ventana temporal). Pero el SVM necesita un **vector de tamaño fijo** por canción. La solución del proyecto es **resumir cada feature en el tiempo** usando su media (`np.mean`) y a veces su desviación estándar (`np.std`):

```
1 mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13) # shape (13, ~1300)
2 mfcc_mean = np.mean(mfcc, axis=1) # 13 numeros: promedio temporal
3 mfcc_std = np.std(mfcc, axis=1) # 13 numeros: cuanto varia en el tiempo
```

- `axis=1` promedia **a lo largo del tiempo**, dejando un valor por banda.
- La **media** captura el “valor típico” de esa propiedad en la canción.
- La **desviación estándar** captura cuánto *varía*: una canción con muchos cambios dinámicos tendrá std alto.

Limitación a mencionar en la defensa

Al promediar en el tiempo **perdemos la estructura temporal** de la canción (estrofa, estribillo, silencios). Para el SVM esto es necesario, pero es justamente lo que una CNN o un modelo recurrente podrían aprovechar.

4. Las 45 features, una por una

Cada canción se convierte en un vector de **45 números**. Esta tabla los agrupa; luego explicamos las más importantes.

Feature	#	Qué describe
MFCC (mean)	13	Forma del espectro → timbre (valor típico)
MFCC (std)	13	Variación del timbre a lo largo de la canción
Chroma	12	Energía en cada una de las 12 notas → armonía
Spectral Centroid (mean/std)	2	“Brillo”: dónde está el centro de masa del espectro
Spectral Rolloff (mean)	1	Frecuencia bajo la cual está el 85 % de la energía
Spectral Bandwidth (mean)	1	“Ancho” del espectro alrededor del centroide
Zero Crossing Rate (mean)	1	Cuántas veces la onda cruza el cero (ruido/-percusión)
Tempo	1	Velocidad estimada en BPM
RMS Energy (mean)	1	Energía/volumen promedio
Total	45	

4.1 MFCC — la feature estrella (análisis en profundidad)

El profesor pidió investigar al menos una feature a fondo. Los **MFCC** (*Mel-Frequency Cepstral Coefficients*) son la mejor candidata: son las más importantes para el modelo y las más interesantes conceptualmente.

4.1.1 ¿Qué problema resuelven?

Queremos describir el **timbre**: eso que hace que una guitarra y un piano tocando la misma nota suenen distinto. El timbre está en la *forma* de la envolvente del espectro (qué armónicos resalta el instrumento), no en la nota en sí.

4.1.2 Cómo se calculan (paso a paso)

1. **Espectrograma** vía STFT (visto antes).
2. **Escala Mel**: las frecuencias se reagrupan según la *escala Mel*, que imita cómo el oído humano percibe el tono. Distinguimos muy bien los graves pero comprimimos los agudos: la escala Mel hace lo mismo.
3. **Logaritmo**: se toma el log de la energía, porque percibimos el volumen de forma logarítmica (decibeles).
4. **DCT** (Transformada Coseno Discreta): comprime todo en unos pocos coeficientes que describen la *forma* de la envolvente espectral. Conservamos los primeros 13 (`n_mfcc=13`).

Interpretación de los coeficientes

- El MFCC 0 se relaciona con la **energía global** del frame.

- Los **coeficientes bajos** (1–4) capturan la forma gruesa del espectro (¿es un sonido “oscuro” o “brillante”?).
- Los **coeficientes altos** (5–12) capturan detalles finos del timbre.
Por eso son tan buenos separando *metal* (distorsión, mucha energía en agudos) de *classical* (timbre suave de cuerdas y maderas).

4.2 Chroma — la armonía y las notas

`chroma_stft` colapsa todas las octavas en las **12 clases de notas** (Do, Do#, Re, ..., Si). Cada uno de los 12 valores indica cuánta energía hay en esa nota a lo largo de la canción. Captura la **armonía**: qué acordes y tonalidades predominan. Es clave para géneros con armonía marcada (jazz, clásica) frente a otros más rítmicos o ruidosos (metal, hiphop).

4.3 Features espectrales — el “color” del sonido

Spectral Centroid (brillo)

Es el “centro de masa” del espectro. Si hay mucha energía en agudos, el centroide es alto y el sonido se percibe *brillante* (metal, pop electrónico). Si predominan los graves, es bajo y suena *oscuro* (clásica, jazz suave).

Spectral Rolloff

La frecuencia por debajo de la cual se acumula el 85 % de la energía. Distingue sonidos con mucho contenido en agudos de los más “apagados”.

Spectral Bandwidth

Cuán “ancho” es el espectro alrededor del centroide. Sonidos ruidosos/distorsionados tienen bandwidth alto; tonos puros, bajo.

4.4 Features temporales — ritmo y energía

Zero Crossing Rate (ZCR)

Cuántas veces por segundo la onda cruza el cero. Es alto en sonidos ruidosos/percusivos (platillos, distorsión, voz fricativa) y bajo en tonos limpios. Útil para detectar percusión.

Tempo (BPM)

`librosa.beat.beat_track` estima la velocidad detectando picos periódicos de energía (los “golpes”). Separa baladas lentas de música bailable.

RMS Energy

La raíz cuadrática media de la amplitud: el “volumen” promedio. Géneros agresivos (metal) tienen RMS alto y constante; la clásica tiene mucha variación dinámica.

4.5 Por qué se normaliza (StandardScaler)

Las features viven en escalas muy distintas: el tempo ronda 60–180, el ZCR está entre 0 y 1, el rolloff llega a miles de Hz. Sin normalizar, el SVM le daría *artificialmente* más importancia a las features con números grandes.

```
1 scaler = StandardScaler()
2 X_train_scaled = scaler.fit_transform(X_train) # media 0, std 1
3 X_test_scaled = scaler.transform(X_test)      # MISMA transformacion
```

Detalle importante

El scaler se ajusta (fit) **solo con el train** y luego se *aplica* al test. Si se ajustara con todo,

habría “data leakage”: el modelo vería información del test durante el entrenamiento. Por eso el scaler se guarda en `models/scaler.pkl` y se reutiliza con el folklore.

5. Modelo 1: SVM (Support Vector Machine)

5.1 Idea intuitiva

Un SVM busca el **hiperplano** que mejor separa dos clases, maximizando el *margen* (la distancia a los puntos más cercanos de cada clase, llamados **vectores de soporte**). Cuanto más ancho el margen, mejor generaliza.

5.2 El kernel RBF: por qué es no lineal

Nuestras 45 features no se separan con líneas rectas. El **kernel RBF** (*Radial Basis Function*) resuelve esto midiendo **similitud por cercanía**:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

Dos canciones con features parecidas (distancia chica) tienen $K \approx 1$; muy distintas, $K \approx 0$. Esto equivale a proyectar los datos a un espacio de dimensión altísima donde *sí* se pueden separar con un hiperplano. El resultado son fronteras **curvas** en el espacio original.

```
1 model = SVC(kernel='rbf', random_state=42, probability=True)
2 model.fit(X_train_scaled, y_train)
```

- `probability=True` permite obtener `predict_proba`: la “confianza” del modelo en cada género. Es lo que usamos para ver qué tan seguro está al clasificar folklore.
- `random_state=42` hace el experimento **reproducible**.

5.3 Multiclase: 10 géneros con un clasificador binario

El SVM es binario por naturaleza. Scikit-learn lo extiende con la estrategia **one-vs-one**: entrena un SVM por cada par de géneros ($\binom{10}{2} = 45$ clasificadores) y vota. El género más votado gana.

5.4 Resultados del SVM (GTZAN)

Género	Precision	Recall	F1
Classical	0.947	0.900	0.923
Jazz	0.731	0.950	0.826
Pop	0.762	0.800	0.780
Metal	0.875	0.700	0.778
Country	0.667	0.700	0.683
Blues	0.619	0.650	0.634
Rock	0.714	0.500	0.588
Reggae	0.611	0.550	0.579
Hiphop	0.478	0.550	0.512
Disco	0.476	0.500	0.488
Accuracy global	~68 %		

Cómo leer estos números

Precision: de todo lo que predije como “X”, cuánto era realmente X. **Recall:** de todo lo que era realmente X, cuánto detecté. **F1:** media armónica de ambos. **Classical** casi perfecto (timbre muy distintivo); **disco/hiphop** se confunden entre sí (beats electrónicos similares). El 68 % es bueno: el azar daría 10 %.

6. El corazón del proyecto: por qué el modelo confunde el folklore

Aquí está la parte que el profesor quiere que entendamos de verdad: **por qué** el modelo “mira” al folklore de cierta forma.

6.1 Experimento A: folklore sin entrenar

El SVM solo conoce 10 géneros. Al darle folklore, **predict** lo fuerza a elegir el “más parecido” según las features. Resultado típico: la mayoría cae en **country**, y algunas canciones con violín o cuerdas en **classical**.

La explicación acústica (clave de la defensa)

El modelo **no entiende de cultura**; solo compara números. El folklore comparte con esos géneros sus **features superficiales**:

- **Folklore → country:** ambos usan **guitarra acústica**, estructuras armónicas simples (pocos acordes → chroma parecido) y tempos medios (90–140 BPM). Los MFCC del timbre de guitarra acústica caen cerca de los del country.
- **Folklore con violín → classical:** el violín produce un timbre de cuerda frotada (MFCC) y un centroide espectral parecidos a los de la música clásica. El modelo “ve” cuerdas y dispara classical.

Conclusión: el modelo captura **similitudes acústicas reales**, pero no el contexto cultural que define al folklore.

6.2 Experimento B: agregar folklore como género nuevo

Ahora agregamos el folklore (en total **70 canciones**) como un género nuevo (#11), reentrenamos y vemos si el modelo es capaz de reconocer canciones de folklore que **no** vio durante el entrenamiento.

6.2.1 La primera versión (ingenua) y su problema

El primer script (4_folklore_training_and_testing.py) apartaba **una sola** canción para test y entrenaba con el resto:

```
1 df_folklore_shuffled = df_folklore.sample(frac=1).reset_index(drop=True)
2 df_folklore_test     = df_folklore_shuffled.iloc[:1]      # 1 cancion no vista
3 df_folklore_train    = df_folklore_shuffled.iloc[1:]      # el resto a entrenar
4 df_combined = pd.concat([df_gtzan, df_folklore_train], ignore_index=True)
```

Por qué esto NO es una evaluación válida

Probar con **una sola** canción no mide nada confiable: el accuracy solo puede dar 0 % o 100 %, y depende enteramente de qué canción tocó al azar (encima con **random_state=None**, cambia en cada corrida). Es una *demonstración ilustrativa* (“mirá, ahora sí predice folklore”), no una *medición*. El número confiable exige **cross-validation**.

6.2.2 La versión correcta: cross-validation 5-fold

El script `4b_folklore_crossvalidation.py` arregla esto. La idea de la **validación cruzada**: partir las 70 canciones de folklore en 5 grupos (*folds*). En cada ronda, 4 grupos van a entrenamiento (junto con GTZAN) y 1 grupo se reserva para test; se rota hasta que **cada canción fue evaluada exactamente una vez** sobre un modelo que no la vio. Así aprovechamos las 70 canciones para entrenar y evaluar, y obtenemos una métrica estable (promedio $\pm$ desvío).

```
1 kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
2 for train_idx, test_idx in kf.split(X_folklore, dummy_y):
3     X_train = np.vstack([X_gtzan, X_folklore[train_idx]]) # GTZAN + folk train
4     y_train = np.concatenate([y_gtzan, ['folklore']*len(train_idx)])
5     scaler.fit(X_train); model.fit(scaler.transform(X_train), ...)
6     pred = model.predict(scaler.transform(X_folklore[test_idx])) # folk no visto
```

6.2.3 Resultados reales

Métrica	Valor
Recall de folklore (5-fold CV)	65.7 % $\pm$ 9.5 %
Canciones reconocidas como folklore	46 / 70
Accuracy global del modelo de 11 clases	68.2 %
<i>¿A dónde van las 24 que falla?</i>	
country	10 (error #1)
blues	4
rock	3
jazz / otros	7

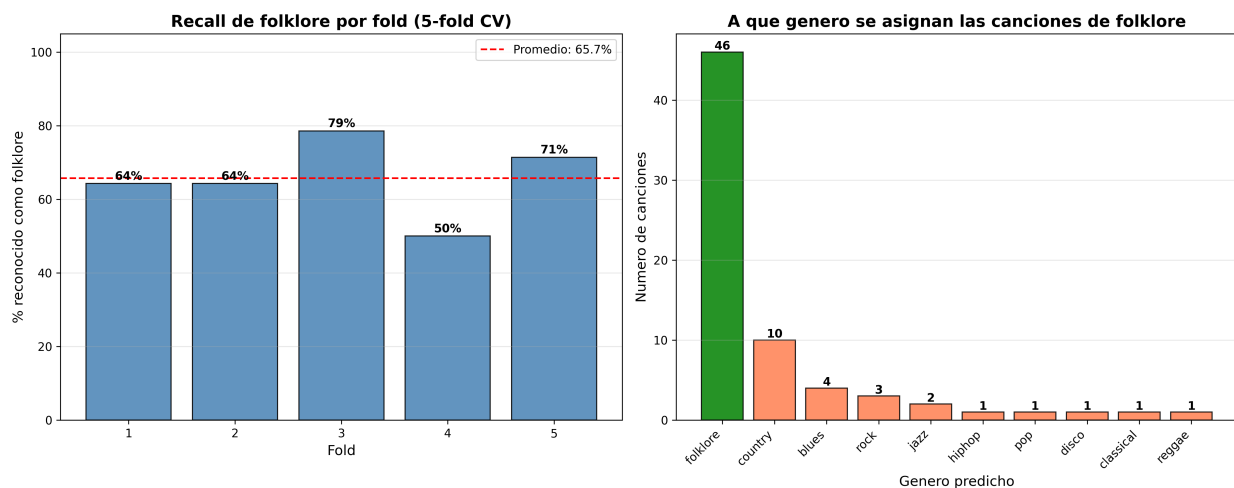


Figura 1: Validación cruzada del folklore. Izquierda: recall por fold (estable alrededor del 66 %). Derecha: a qué género se asigna cada canción — la mayoría acierta “folklore”, y los errores caen sobre todo en *country*.

Qué demuestra esto (y cómo conecta con el Experimento A)

1. **Las features sí sirven:** con 70 ejemplos el modelo reconoce el folklore $\sim 66\%$ de las veces, muy por encima del azar ($1/11 \approx 9\%$). El problema del Experimento A no eran las features, era la **falta de ejemplos**.

2. **Confirma la historia acústica:** cuando el modelo *falla*, el error más frecuente sigue siendo **country** (10 de 24), seguido de blues y rock — todos géneros de **guitarra acústica**. Es exactamente lo que predecía el Experimento A: el folklore “suena” acústicamente parecido al country.
3. **El número es honesto:** $65.7\% \pm 9.5\%$ es una métrica promediada sobre las 70 canciones, no una corrida con suerte.

Por qué el folklore es difícil incluso así

Un 66 % es bueno pero no altísimo, y tiene una explicación de fondo: el folklore argentino es **muy heterogéneo** (una chacarera, una zamba y un chamamé suenan distinto entre sí). Lo estamos forzando a ser *una* clase. Separarlo en subgéneros, o sumar más canciones, subiría el recall. Es la principal línea de trabajo futuro.

7. Modelo 2: Red Neuronal (MLP)

Para cumplir con el requisito de usar **al menos dos modelos**, se entrena un Perceptrón Multicapa (MLP) con Keras sobre las *mismas* 45 features.

```

1 model = keras.Sequential([
2     layers.Dense(128, activation='relu', input_shape=(45,)),
3     layers.Dropout(0.3),
4     layers.Dense(64, activation='relu'),
5     layers.Dropout(0.3),
6     layers.Dense(32, activation='relu'),
7     layers.Dense(10, activation='softmax')    # 10 generos
8 ])

```

7.1 Anatomía de la red

Capa de entrada (45)

Un nodo por feature.

Capas ocultas (128 → 64 → 32)

Cada Dense aprende combinaciones de la capa anterior. **ReLU** ($\max(0, x)$) introduce la no linealidad que permite aprender patrones complejos.

Dropout (0.3)

En cada paso de entrenamiento “apaga” al azar el 30 % de las neuronas. Esto **evita el sobreajuste**: la red no puede depender de una sola neurona y aprende representaciones más robustas.

Capa de salida (10, softmax)

Softmax convierte las 10 salidas en una distribución de probabilidad que suma 1: “75 % rock, 15 % metal...”.

7.2 Cómo aprende

- **Loss = categorical_crossentropy**: mide cuán lejos está la probabilidad predicha de la respuesta correcta (one-hot). Entrenar = minimizar esta pérdida.
- **Optimizer = adam**: ajusta los pesos por descenso de gradiente con tasa de aprendizaje adaptativa.
- **50 epochs**: pasa 50 veces por todo el train. `validation_split=0.2` reserva 20 % para vigilar el sobreajuste durante el entrenamiento.

7.3 Comparación cualitativa

	SVM (RBF)	Red Neuronal (MLP)
Datos ideales	Pocos/medianos	Muchos
45 features	Muy cómodo	Funciona, puede sobreajustar
Interpretabilidad	Media (vectores de soporte)	Baja (caja negra)
Ajuste de hiperparámetros	Pocos (C , γ)	Muchos (capas, dropout, lr)
Determinismo	Determinista (semilla fija)	Ruidoso (init aleatoria)

7.4 Comparación justa (mismo dataset y mismo split)

Cuidado con las comparaciones tramposas

El script original de la red reportaba **77 % vs 68 % del SVM**, pero esa comparación era **inválida**: la red se medía solo sobre GTZAN (10 clases, sin folklore) y con un split **sin estratificar**, mientras que el SVM usaba un split estratificado sobre datos distintos. Peras con manzanas.

El script `7_model_comparison.py` hace la comparación **correcta**: entrena *ambos* modelos sobre el **mismo** dataset ampliado (GTZAN + folklore, 11 clases), con el **mismo** split estratificado 80/20 y el mismo `StandardScaler`.

Métrica	SVM	Red Neuronal
Accuracy global	68.2 %	65–68 % (varía)
Folklore — precision	75.0 %	~53 %
Folklore — recall	64.3 % (9/14)	64.3 % (9/14)
Folklore — f1-score	69.2 %	58–72 % (varía)

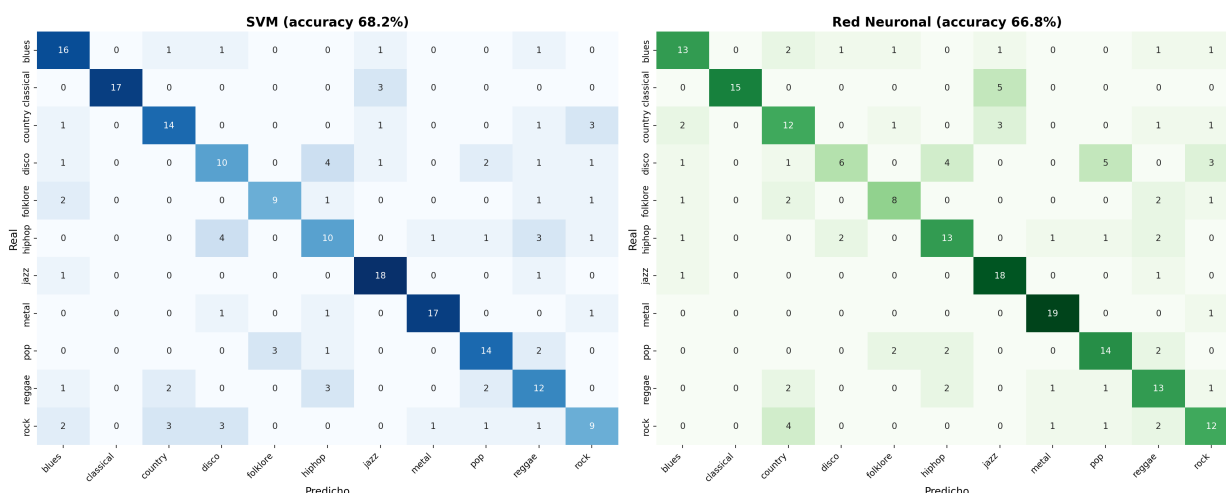


Figura 2: Matrices de confusión de ambos modelos sobre el mismo conjunto de test de 11 clases. Rendimiento muy parecido; el folklore se reconoce de forma similar en los dos.

Conclusión de la comparación

Enfrentados de forma justa, **SVM y red neuronal quedan parejos (~66–68 %)**: la red **no** es mejor. Es más, el SVM gana levemente y de forma *estable*, mientras que la red varía

entre corridas por su inicialización aleatoria. La razón de fondo: con un dataset **chico** (1000 canciones) y features ya “destiladas” por **librosa**, un modelo más complejo no tiene de dónde sacar ventaja. **Una buena representación de los datos (las features) suele importar más que la complejidad del modelo.**

8. Modelo 3 (descartado): CNN sobre espectrogramas

A diferencia de los anteriores, la CNN **no usa las 45 features**: trabaja directamente sobre el **mel-espectrograma** (una imagen frecuencia $\times$ tiempo de 128×130). La idea es tratar la clasificación de audio como **clasificación de imágenes**.

- Las capas Conv2D aprenden **sus propios filtros** (texturas del espectrograma) en lugar de usar features diseñadas a mano.
- MaxPooling reduce la resolución; BatchNormalization y Dropout estabilizan y regularizan.
- Conserva la **estructura temporal** que el promedio mean/std destruía.

Por qué se descartó (en principio)

La CNN necesita **muchos más datos** y cómputo para superar al SVM; con GTZAN (y solo 100 clips/género) tiende a sobreajustar y no aporta una mejora clara que justifique la complejidad. Queda documentada como **línea de trabajo futuro**, no como resultado principal.

9. Preguntas probables en la defensa (y cómo responderlas)

¿Qué hace exactamente `librosa.load`?

Muestrea el audio a 22050 Hz y devuelve un arreglo de amplitudes y el sample rate **sr**. Tomamos 30 s para que todas las canciones sean comparables.

¿Qué es un MFCC en una frase?

Un conjunto de coeficientes que describen la **forma de la envolvente espectral** en escala perceptual (Mel), es decir, el **timbre** del sonido.

¿Por qué promedian las features en el tiempo?

Porque el SVM necesita un vector de tamaño fijo. Resumimos cada feature temporal con su media y desviación estándar, asumiendo el costo de perder la estructura temporal.

¿Por qué folklore $\rightarrow$ country?

Comparten timbre de guitarra acústica (MFCC), armonía simple (chroma) y tempo medio. El modelo compara números, no cultura.

¿Por qué normalizan?

Para que features con escalas grandes (tempo, rolloff) no dominen sobre las chicas (ZCR). El scaler se ajusta solo con el train para evitar data leakage.

¿Por qué SVM y no solo red neuronal?

Con un dataset chico y features ya informativas, el SVM generaliza muy bien con pocos hiperparámetros. La red neuronal se incluye para comparar y necesita más datos.

¿Es robusto el Experimento B?

El primer script no (test de 1 canción). Por eso hicimos la versión correcta (4b) con **cross-validation 5-fold** sobre las 70 canciones: recall de folklore **65.7% $\pm$ 9.5%**, una métrica estable y promediada, no una corrida con suerte.

¿Qué mejorarían?

Más folklore (50–100), separarlo en subgéneros (chacarera, zamba, chamamé), cross-validation, y explorar features temporales o una CNN con más datos.

Fin de la guía de estudio. ¡Éxitos en la defensa!